

Содержание

1	Инструкция пользователя	2
1.1	Формат входных данных	2
1.2	Запуск	3
1.3	Пример входных данных	3
1.4	Пример выходных данных	3
2	Выбранные технологии	5
2.1	Язык программирования: Haskell	5
2.2	Система сборки Stack	6
2.3	Библиотеки	6
2.4	SMT-решатель Z3	6
3	Архитектура	7
3.1	Общая схема	7
3.2	Фаза 1: доказательство (неаннотированное)	7
3.2.1	Классификация	7
3.2.2	Поиск вывода в исчислении SAgrow	9
3.3	Фаза 2: аннотация	10
3.3.1	Аннотирование SAgrow	11
3.3.2	Аннотирование классификации	11
3.3.3	Редукция итогового термина	12
3.4	Фаза 3: контрмодель	13
4	Примеры использования	13
4.1	Валидная формула: $a \Rightarrow a$	13
4.2	Валидная формула: modus ponens	14
4.3	Невалидная формула: схема Пирса	14
4.4	Невалидная формула: исключённое третье	15
4.5	Невалидная формула: закон де Моргана для дизъюнкции	15
4.6	Сложный валидный пример	16
4.7	Сложный валидный пример: вложенная схема Пирса	16

1 Инструкция пользователя

Программа `diploma-exe` проверяет, является ли заданная формула теоремой интуиционистской пропозициональной логики (IPL). Точка входа — функция `test2` в `app/Main.hs`:

```
-- snippet from source
test2 = do
  args <- getArgs
  formulaFile <- case args of
    [path] -> pure path
    _ -> die "Usage: diploma-exe <formula-file>"
  formulaText <- readFile formulaFile
  let formula = parseFormulaWithError $ trim formulaText

  IncrementalSolver context solver <- IncrementalSolver.newSolver
  result <- ProverR.proveR context solver formula
  case result of
    ProverR.Invalid counterModel -> ...
    ProverR.Valid (plaiidt, clausificationHistory) -> do
      let (annotatedCArrowNodes, st) =
          runState (annotateCArrowNodes carrowNodes) newEnvironment
      let (annotatedClausificationNodes, _) =
          runState (annotateClausificationNodes lastCArrowSeq
              clausificationNodes) st
      ...
```

Листинг 1: Точка входа: `test2` в `app/Main.hs`

При положительном ответе строится аннотированное дерево доказательства; при отрицательном — контрмодель Kripke.

1.1 Формат входных данных

Вход — текстовый файл с одной формулой IPL. Синтаксис задаётся модулем `Parser.hs` (библиотека `Parsec`). Грамматика построена комбинаторами с приоритетом операторов (см. ниже).

```
-- snippet from source
implications = do ds <- disjunctions
  is' <- implications'
  return $ Foldable.foldr1 implication (ds:is')

disjunctions = ... -- snippet from source
conjunctions = ... -- snippet from source
negations    = do negationSign
  flip implication bottom <$> negations
  <|> unit
```

Листинг 2: Грамматика парсера формул (`Parser.hs`)

- **Переменные** — последовательности латинских букв: `A`, `a`, `foo`.

- **Конъюнкция** — $\wedge$ между операндами, с пробелами: $A \wedge B$.
- **Дизъюнкция** — $\vee$: $A \vee B$.
- **Импликация** — $\Rightarrow$, правоассоциативна: $A \Rightarrow B \Rightarrow C$ означает $A \Rightarrow (B \Rightarrow C)$.
- **Отрицание** — унарный префикс $\neg$: $\neg A$ означает $A \Rightarrow _ _$.
- **Ложь** — $_ _$.
- **Скобки** — $(\dots)$ для группировки.

Приоритет (от слабого к сильному): импликация, дизъюнкция, конъюнкция, отрицание, атом/скобки.

Пробелы в начале и конце файла игнорируются.

1.2 Запуск

Сборка и запуск через Stack:

```
stack build
stack exec diploma-exe -- path/to/formula.txt
```

Листинг 3: Сборка и запуск программы

Результат выводится в стандартный поток вы stdout.

1.3 Пример входных данных

Формула тождества (импликация переменной в саму себя):

```
a => a
```

Листинг 4: Пример входной формулы: $a \Rightarrow a$

Эквивалентный пример в файле `examples/formulas/valid-identity.txt`:

```
A => A
```

Листинг 5: Файл `valid-identity.txt`

Другие примеры лежат в каталоге `examples/formulas/` (валидные, невалидные и сложные формулы).

1.4 Пример выходных данных

Для формулы $a \Rightarrow a$ (или $A \Rightarrow A$) программа выводит историю классификации, результат поиска в исчислении SAT_Q, аннотированные секвенции и графы DOT.

Фрагмент вывода (сокращён):

```

R:  X: ((A => A) => $) |- $ by ImplImpliesFlat
R: {} X: {((A => A) => $)} U: {} |- $ by AsIntuit
R: {} X: {} U: {((A => A) => $)} |- $ by AsImpl
Valid
CARROW:
Ds: [R: ... A: ... |- $ ...] {Ax: [R: ... A: A ... |- A ...]}
...
CLAUSIFICATION:
R: (A => $) ~ $0
X: ((A => A) => $) ~ $3
|-
$
($3I)
...
digraph Proof { ... }

```

Листинг 6: Фрагмент вывода для $A \Rightarrow A$ (сокращён)

Секции вывода:

- **история клозификации** — плоские секвенции до CARROW;
- **Valid** / контрмодель;
- **CARROW** — шаги CPL0/CPL1 с классическими поддеревьями LJT;
- **CLAUSIFICATION** — аннотированные секвенции с λ -термами;
- **digraph** — графы Graphviz (LJT, объединённое дерево Proof, контрмодель).

В аннотированной секвенции зоны обозначаются так:

- **R** — flat-клозы (pesidue);
- **X** — импликационные клозы;
- **U** — неклозифицированные формулы;
- **A** — классические допущения (в LJT);
- после **/-** — цель (**AnnotatedGoal**): формула, редуцированный λ -терм и локальные подстановки:

```

-- snippet from source
data AnnotatedGoal = AnnotatedGoal {
  goalTerm          :: Term,
  goalSubstitutions :: [(String, Term)]
}

formatGoal goal =
  let (goalFormula, AnnotatedGoal term substitutions) = Map.findMin goal
  in formulaToString goalFormula ++ "\n" ++ show (reduce term)
    ++ unlines (map (\(n, t) -> n ++ " / " ++ show t) substitutions)

goalPreserveTerm goal =
  let (atom, AnnotatedGoal term _) = Map.findMin goal
  in singletonGoal atom term -- snippet from source

```

Листинг 7: Тип AnnotatedGoal и форматирование цели

2 Выбранные технологии

2.1 Язык программирования: Haskell

Haskell выбран как язык с сильной статической типизацией и удобной поддержкой алгебраических типов данных. Результат проверки описывается GADT ValidationResult:

```

-- snippet from source
data ValidationResult where
  Valid :: ([ (PlainSequent, PlainSequent, CArrowRule)],
            [ (PlainSequent, ClausificationRule)]) -> ValidationResult
  Invalid :: CounterModel -> ValidationResult

```

Листинг 8: GADT ValidationResult (ProverR.hs)

Деревья доказательств (LJT, CArrow), секвенции и правила вывода естественно описываются ADT и GADT (AnnotatedLJTNode). Монада State используется для аннотации термов с окружением свежих переменных (Environment):

```

-- snippet from source
data Environment = Environment {
  cache  :: Map PlainFormula Term,
  vcount :: Int
}

getTermFromEnvironment :: Formula a => a -> State Environment Term
getTermFromEnvironment formula = state newState
  where
    newState env@(Environment cache vcount) =
      case Map.lookup (Formula.plain formula) cache of
        Just term -> (term, env)
        Nothing   -> (Var $ "$" ++ show vcount,
                      env { cache = ..., vcount = vcount + 1 })

```

Листинг 9: Окружение Environment для аннотации термов

Чистота функций упрощает разделение фаз: клонификация и поиск вывода не зависят от аннотации; последняя выполняется отдельно по сохранённой истории.

2.2 Система сборки Stack

Stack фиксирует версию компилятора GHC (9.6.7) и управляет зависимостями (`stack.yaml`, `package.yaml`). Это обеспечивает воспроизводимую сборку на разных машинах без ручной настройки Cabal.

2.3 Библиотеки

- **Parsec** — парсер комбинаторов для входного синтаксиса формул. Альтернатива (Happy/Alex) избыточна для небольшой грамматики.
- **containers** — Map и Set для секвенций, окружений термов и контрмоделей.
- **mtl** — монада State в модулях Clausify, Proof, ClassicSeqProver.
- **haskell-z3** (форк IagoAbal/haskell-z3) — Haskell-биндинги к Z3. Z3 используется не как основной доказатель, а как SAT/SMT-орасле для проверки выполнимости и извлечения *unsat core*.

Ограничение биндингов: API Z3 не предоставляет прямого отображения `FuncDecl` → AST для всех констант модели. При интерпретации модели (`World.fromModel`) необходимо знать, какому атому формулы соответствует каждая константа в модели Z3. Поэтому в `ProverR.prover` строятся три отображения (`varToASTMap`, `astToVarMap`, `funcDeclToASTMap`):

```
-- snippet from source
varToASTMap <- sequence $ Map.fromSet (Formula.atomToAST context) vars
let varToAST = (!) varToASTMap
let astToVar = (!) (Map.fromList $ map swap $ Map.toList varToASTMap)

funcDeclToASTMap <-
  Map.fromList <$>
  mapM (\ast -> (,ast) <$> (Z3.getAppDecl context =<< Z3.toApp context ast)
    )
  (Map.elems varToASTMap)

let funcDeclToAST = (!) funcDeclToASTMap
```

Листинг 10: Отображения атомов и `FuncDecl` → AST в `proveR`

2.4 SMT-решатель Z3

Z3 применяется инкрементально (`IncrementalSolver`). Flat-клозы кодируются как импликации и добавляются в solver; проверка цели и извлечение *unsat core* реализованы в `satProve`:

```

-- snippet from source
addClause context solver (FlatClauseFormula cs ds _) = do
  and <- Z3.mkAnd context cs
  or  <- Z3.mkOr context ds
  implication <- Z3.mkImplies context and or
  Z3.solverAssertCnstr context solver implication

satProve funcDeclToAST context solver adds goal = do
  goalAST <- Z3.mkNot context goal
  checkResult <- Z3.solverCheckAssumptions context solver (goalAST : adds)
  case checkResult of
    Z3.Sat    -> No <$> (World.fromModel funcDeclToAST context
                        =<< Z3.solverGetModel context solver)
    Z3.Unsat  -> Yes . List.delete goalAST
                  <$> Z3.solverGetUnsatCore context solver

```

Листинг 11: Кодирование flat-кловов и проверка `satProve`

При `Sat` модель интерпретируется через `World.fromModel` (см. снippet ниже в разделе о контрмодели).

Доказательства `Z3` в sequent-форму напрямую не интерпретируются, поэтому реализован собственный прувер: поиск по правилам `CArrow` (CPL0/CPL1) с классическими поддеревьями `LJT` и последующей λ -аннотацией по Curry–Howard.

3 Архитектура

3.1 Общая схема

Файл с формулой	→ <code>Parser</code>
→	<code>Clausify.clausify</code> (фаза 1a)
→	<code>ProverR.proveR</code> + <code>Z3</code> (фаза 1b)
→	<code>Proof.annotateCArrowNodes</code> (фаза 2a)
→	<code>Proof.annotateClausificationNodes</code> (фаза 2b)
→	Вывод + <code>ProofDot</code> / <code>CounterModel</code>

3.2 Фаза 1: доказательство (неаннотированное)

3.2.1 Клозификация

Исходная формула F преобразуется в задачу доказательства $F \Rightarrow \$$, где $\$$ — свежий атом-цель.

Вводится тип `UnclassifiedPlainSequent` (и родственные варианты `IntuitPlainSequent`, `ClassicPlainSequent`):

```
-- snippet from source
data PlainSequent
  = ClassicPlainSequent {
    plainFlats      :: [FlatClauseFormula Atom],
    plainAssumptions :: [Atom],
    plainGoal       :: Atom
  }
  | IntuitPlainSequent {
    plainFlats :: [FlatClauseFormula Atom],
    plainImpls :: [ImplClauseFormula Atom],
    plainGoal  :: Atom
  }
  | UnclassifiedPlainSequent {
    plainFlats      :: [FlatClauseFormula Atom],
    plainImpls      :: [ImplClauseFormula Atom],
    plainUnclassified :: [PlainFormula],
    plainGoal       :: Atom
  }
}
```

Листинг 12: Типы секвенций PlainSequent (Sequent.hs)

Из одной секвенции всегда порождается одна следующая; история — список пар (PlainSequent, ClausificationRule).

Правила клозификации (Clausify.hs, Proof.hs):

```
-- snippet from source
data ClausificationRule =
  MakeImpl PlainFormula PlainFormula |
  LeftDs PlainFormula [PlainFormula] |
  RightCs PlainFormula [PlainFormula] |
  RightImpl PlainFormula PlainFormula |
  Aliasing PlainFormula PlainFormula [PlainFormula] |
  ImplImpliesFlat (ImplClauseFormula Atom) (FlatClauseFormula Atom) |
  AsFlat PlainFormula (FlatClauseFormula Atom) |
  AsImpl (ImplClauseFormula Atom) |
  AsIntuit
```

Листинг 13: Перечисление правил клозификации (Proof.hs)

Основной цикл clausifyLoopS и запись истории через putSequent:

```

-- snippet from source
clausifyLoopS s@(UnclassifiedPlainSequent f i (nph : npt) g) = do
  case Clause.flatClauseFromFormula nph of
    Just clause -> do
      putSequent (s, AsFlat nph clause)
      clausifyLoopS (UnclassifiedPlainSequent (clause:f) i npt g)
    Nothing -> case Clause.implClauseFromFormula nph of
      Just clause -> do
        putSequent (s, AsImpl clause)
        clausifyLoopS (UnclassifiedPlainSequent f (clause:i) npt g)
      Nothing -> do
        (clausified, rule) <- clausifyS nph
        putSequent (s, rule)
        clausifyLoopS (UnclassifiedPlainSequent f i (clausified ++ npt) g)

clausifyLoopS s@(UnclassifiedPlainSequent f i [] g) = do
  putSequent (s, AsIntuit)
  return (IntuitPlainSequent f i g)

putSequent seq = ST.state (\st -> ((), BF.second (seq:) st))

```

Листинг 14: Цикл `clausifyLoopS` и запись истории (`Clausify.hs`)

Реализация правил преобразования формул в `clausifyS`:

```

-- snippet from source
clausifyS f@(Implication [Disjunction ds, v@(Atom _)]) = do
  let newFormulas = map ('implication' v) ds
  return (newFormulas, LeftDs f newFormulas)

clausifyS f@(Implication [v@(Atom _), Conjunction cs]) = do
  let newFormulas = map (implication v) cs
  return (newFormulas, RightCs f newFormulas)

clausifyS f@(Implication (x : y : z : is)) = do
  let newFormula = Implication (conjunction x y : z : is)
  return ([newFormula], RightImpl f newFormula)

clausifyS x = do
  let newFormula = implication top x
  return ([newFormula], MakeImpl x newFormula)

```

Листинг 15: Преобразования формул в `clausifyS`

3.2.2 Поиск вывода в исчислении `CArrow`

После клонификации получается `IntuitPlainSequent`. Правила `CArrow`:

```
-- snippet from source
data CArrowRule = CPL0 | CPL1 (FlatClauseFormula Atom) (ImplClauseFormula Atom)

data CArrowNode = CArrowNode {
  carrowRule      :: CArrowRule,
  classicSequent  :: PlainSequent,
  implSequent     :: PlainSequent
}
```

Листинг 16: Правила CArrow: CPL0 и CPL1

Цикл `proveR'` реализует CPL0 и CPL1:

```
-- snippet from source
proveR' rootSeq@(IntuitPlainSequent r x g) plainDt = do
  result <- IncrementalSolver.satProve funcDeclToAST context solver [] goalAST
  case result of
    IncrementalSolver.Yes a ->
      return $ Valid ((rootSeq, ClassicPlainSequent r (map astToVar a) g, CPL0)
                     :plainDt, history)
    IncrementalSolver.No m -> do
      result <- proveR'' implsAST (CounterModel.newCounterModel m)
      case result of
        Left (assumptions, impl) -> do
          let newClause = FlatClauseFormula
              (List.delete (a_ impl) assumptions) [c_ impl] ...
          IncrementalSolver.addClause context solver newClause
          proveR' (IntuitPlainSequent (newClauseFormula:r) x g)
              ((rootSeq, ClassicPlainSequent r (map astToVar assumptions)
              (astToVar $ b_ impl), CPL1 newClauseFormula learnedClauseFormula)
              :plainDt)
        Right counterModel -> return $ Invalid counterModel
```

Листинг 17: Цикл `proveR'`: реализация CPL0 и CPL1

CPL0 Проверка: из flat-теории следует ли \$? Вызывается `satProve` с пустыми допущениями. При **Unsat** `unsat core` даёт классическую секвенцию $\Gamma \vdash \$$.

CPL1 Если CPL0 дал **Sat**, строится контрмодель. Выбирается `impl`-кюз, где a истинно, а b ложно. Проверяется $\Gamma, a \vdash b$. При **Unsat** из `core` строится новый flat-кюз, добавляется в Z3.

История CArrow: список `(PlainSequent_intuit, PlainSequent_classic, CArrowRule)`. Если все `impl`-кюзы исчерпаны без CPL0 — **Invalid**, возвращается `CounterModel`.

3.3 Фаза 2: аннотация

Аннотация выполняется *после* успешного поиска вывода и опирается на сохранённые истории. История кюзификации записывается при построении через `putSequent` в обратном хронологическом порядке (новый шаг добавляется в начало списка). При аннотации этот список обходится с головы, то есть правила применяются в обратном порядке относительно прямой кюзификации.

3.3.1 Аннотирование CArrow

`annotateCArrowNodes` обрабатывает цепочку CPL0/CPL1; на CPL0 сначала строится LJT-доказательство классической секвенции:

```
-- snippet from source
annotateCPL0 (CArrowNode CPL0 cseq iseq@(IntuitPlainSequent _ impls _)) = do
  let provedCseq = proveLJT cseq
  annotatedClassic <- annotateLJTNode provedCseq
  let (ClassicSequent flats assumptions goal) = rootLJT annotatedClassic
  implAnnotations <- mapM getTermFromEnvironment impls
  return (IntuitSequent {
    flats = flats,
    impls = Map.fromList $ zip impls implAnnotations,
    goal = goal
  }, annotatedClassic)
```

Листинг 18: Аннотирование шага CPL0 в `annotateCArrowNodes`

Правила LJT (`ClassicSeqProver.hs`):

```
-- snippet from source
data LJTNode =
  Axiom PlainSequent |
  Cs PlainSequent LJTNode (FlatClauseFormula Atom, FlatClauseFormula Atom, Atom)
  |
  Ds PlainSequent [LJTNode] (FlatClauseFormula Atom)

proveLJT pseq =
  case applyAxiom pseq of
    Just () -> Axiom pseq
    Nothing -> case applyCs pseq of
      Just (newPseq, keyF, newF, atom) -> Cs pseq (proveLJT newPseq) (keyF, newF
, atom)
      Nothing -> case applyDs pseq of
        Just (newPseqs, keyF) -> Ds pseq (map proveLJT newPseqs) keyF
        Nothing -> error "Wrong sequent"
```

Листинг 19: Дерево вывода LJT (`ClassicSeqProver.hs`)

1. **Ax** — цель уже в допущениях (`Axiom`).
2. **Cs** — сокращение по conjunct-side flat-клоза.
3. **Ds** — разбиение по disjunct-side flat-клоза.
4. На CPL1: по классическому доказательству $\Gamma \vdash b$ строится λ -терм с `Case`, `Proj`, `Insert`, `Sum`.

3.3.2 Аннотирование клозификации

`annotateClausificationNodes` обходит историю клозификации (в обратном порядке относительно прямого разбиения):

```
-- snippet from source
annotateClausificationNodes seq [] = return [seq]
annotateClausificationNodes seq (h:t) = do
  annotated      <- annotate seq h
  othersAnnotated <- annotateClausificationNodes annotated t
  return (seq : othersAnnotated)
```

Листинг 20: Обход истории в `annotateClausificationNodes`

Особенность **AsFlat**: атом превращается в терм с конструкторами `Sum`, `Proj`, `Abstraction`:

```
-- snippet from source
annotate seq (ClausificationNode (AsFlat f c) _) = do
  let f'Term = fs ! c
  fTerm <- getNewTermFromEnvironment
  substitution <- case f of
    (Atom _) ->
      Abstraction [varName capture] $ Application [Sum 1, fTerm]
    (Implication [Atom _, Atom _]) ->
      Abstraction [varName capture] $
        Application [Sum 1, Application [fTerm, Application [Proj 1, capture]]]
    (Implication [Atom _, Disjunction ds]) -> ...
    (Implication [Conjunction cs, Atom _]) -> ...
  return seq {
    flats      = Map.delete c fs,
    unclausified = Map.insert f fTerm ucs,
    goal       = applyLocalGoalSubstitution (varName f'Term) substitution g
  }
```

Листинг 21: Аннотирование правила `AsFlat`

Для `AsImpl` и `AsIntuit` подстановки в `goal` сбрасываются через `goalPreserveTerm` (см. листинг 7).

3.3.3 Редукция итогового терма

Функция `Term.reduce` нормализует итоговый λ -терм:

```
-- snippet from source
reduce (Application ts) =
  case map reduce ts of
    (Abstraction [c] body : arg : t) ->
      reduce $ Application (substitute c arg body : t)
    (Insert 1 : Id : arg : t) ->
      reduce $ Application (Product [arg] : t)
    (Proj i : Product ts : t) ->
      reduce $ Application (ts !! (i - 1) : t)
    (Case cs : Application [Sum i, arg] : t) ->
      reduce $ Application (substitute (varName capture) arg body : t)
    ...
```

Листинг 22: Нормализация λ -термов: `Term.reduce`

3.4 Фаза 3: контрмодель

При Invalid возвращается CounterModel:

```
-- snippet from source
data CounterModel = CounterModel {
  reachableTo    :: Map Int [Int],
  reachableFrom  :: Map Int [Int],
  worlds         :: Map Int World
}

selectWorld counterModel impl = ...
  -- snippet from source

counterModelToDot context counterModel = do
  ...
  return $ unlines
    ( "digraph CounterModel {"
    : "  graph [rankdir=BT];"
    : nodeLines ++ edgeLines ++ ["}"] )
```

Листинг 23: Структура контрмодели CounterModel

Миры строятся из моделей Z3 через World.fromModel:

```
-- snippet from source
fromModel funcDeclToAST context model = do
  constDecls <- Z3.getConsts context model
  interps    <- catMaybes <$> mapM (Z3.getConstInterp context model) constDecls
  interpValues <- mapM (Z3.getBool context) interps
  let astToValues = zip (map funcDeclToAST constDecls) interpValues
  return World {
    model    = model,
    consts   = Set.fromList [ast | (ast, True) <- astToValues]
  }
```

Листинг 24: Построение мира из модели Z3: World.fromModel

4 Примеры использования

4.1 Валидная формула: $a \Rightarrow a$

Вход:

```
a => a
```

Листинг 25: Входная формула: $a \Rightarrow a$

Команда:

```
stack exec diploma-exe -- examples/formulas/valid-identity.txt
```

Листинг 26: Запуск на примере valid-identity.txt

(в файле используется имя A , результат тот же.)

Результат: `Valid`. Дерево `CArrow` содержит `CPL0` (классическое поддерево $Ds \rightarrow Ax$) и `CPL1`; клозификация — `AsIntuit`, `AsImpl`, `ImplImpliesFlat`.

Результирующий терм:

```
($3I)
```

Листинг 27: Редуцированный λ -терм для $A \Rightarrow A$

Фрагмент объединённого графа `digraph Proof`:

```
ca0 [label="CPL0\n..."]
ca0 -> ca0_ljt_1 [label="left"];
ca3 [label="CPL1\n..."]
ca3 -> ca3_ljt_4 [label="left"];
ca3 -> ca0 [label="right"];
c11 -> ca3 [label="ImplImpliesFlat"];
c12 -> c11 [label="AsIntuit"];
c13 -> c12 [label="AsImpl"];
```

Листинг 28: Фрагмент графа `digraph Proof` для $a \Rightarrow a$

4.2 Валидная формула: *modus ponens*

Вход (`examples/formulas/valid-modus-ponens.txt`):

```
A /\ (A => B) => B
```

Листинг 29: Входная формула *modus ponens*

Результат: `Valid`, аннотированное дерево с секвенциями в зонах R , X , U и λ -термами в цели.

Результирующий терм:

```
($28(\$17.((P(2)$17)(P(1)$17))))
```

Листинг 30: Редуцированный λ -терм для *modus ponens*

4.3 Невалидная формула: схема Пирса

Вход (`examples/formulas/invalid-peirce.txt`): $((A \Rightarrow B) \Rightarrow A) \Rightarrow A$

Полученные клозы (после клозификации):

- R (flat): $(B \Rightarrow 1), ((0 \wedge 1) \Rightarrow A)$
- X (impl): $((A \Rightarrow B) \Rightarrow 1), ((0 \Rightarrow A) \Rightarrow \$)$

Результат: контрмодель Kripke (рис. 1); формула невалидна (не выводится `Valid`).

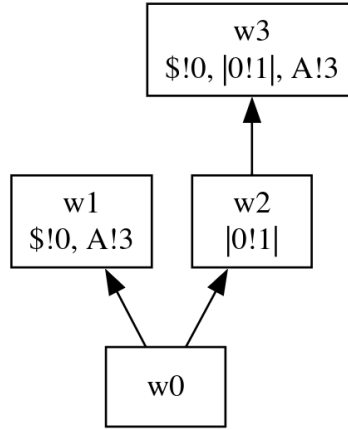


Рис. 1: Контрмодель Крике для схемы Пирса

4.4 Невайдная формула: исключённое третье

Вход (examples/formulas/invalid-excluded-middle.txt): $A \vee \neg A$

Полученные клозы (после клозификации):

- **R** (flat): $(A \Rightarrow \$)$
- **X** (impl): $((A \Rightarrow _ | _) \Rightarrow \$)$

Результат: контрмодель Крике (рис. 2).

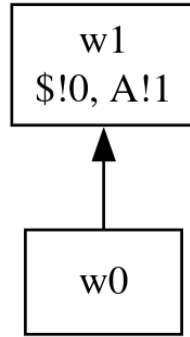


Рис. 2: Контрмодель Крике для формулы $A \vee \neg A$

4.5 Невайдная формула: закон де Моргана для дизъюнкции

Вход (examples/formulas/invalid-de-morgan-or.txt): $\neg(A \wedge B) \Rightarrow (\neg A \vee \neg B)$

Полученные клозы (после клозификации):

- **R** (flat): $(_ | _ \Rightarrow 1)$, $((0 \wedge A \wedge B) \Rightarrow _ | _)$
- **X** (impl): $((B \Rightarrow _ | _) \Rightarrow 1)$, $((A \Rightarrow _ | _) \Rightarrow 1)$, $((0 \Rightarrow 1) \Rightarrow \$)$

Результат: контрмодель Крике (рис. 3); формула невайдная (не выводится Valid).

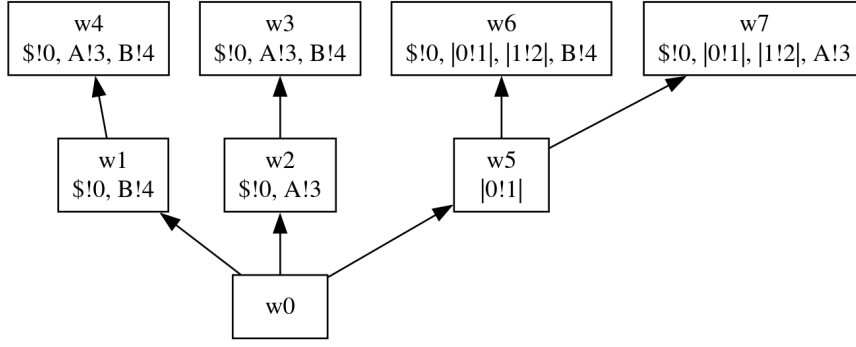


Рис. 3: Контрмодель Крипке для $\neg(A \vee B) \Rightarrow (\neg A \wedge \neg B)$

4.6 Сложный валидный пример

Вход (examples/formulas/complex-and-impl-chain.txt):

```
((a /\ (b /\ c) /\ d) => c) => a) => a
```

Листинг 31: Сложная валидная формула с цепочкой импликаций

Результат: Valid, длинная цепочка клозификации (AsFlat, RightCs, AsImpl, Aliasing, RightImpl, ...) и соответствующий граф digraph Proof.

Результирующий терм:

```
($62(\$25.($25(\$40.(P(4)$40))))))
```

Листинг 32: Редуцированный λ -терм для сложной формулы

4.7 Сложный валидный пример: вложенная схема Пирса

Вход (examples/formulas/complex-peirce-nested.txt):

```
((((a => b) => a) => a) => b) => b
```

Листинг 33: Вложенная схема Пирса

Результат: Valid. Клозификация включает несколько шагов Aliasing, RightImpl, AsFlat, AsImpl и три применения ImplImpliesFlat; доказательство в CArrow использует CPL0 и CPL1.

Результирующий терм:

```
($58(\$28.($28(\$37.($37(\$43.($28K($43)))))))
```

Листинг 34: Редуцированный λ -терм для вложенной схемы Пирса